

实验单元2： XSS 攻击

2.1 实训说明

2.1.1 实训目的

- 1.掌握反射型 XSS 的原理和利用方法
- 2.掌握存储型 XSS 的原理和利用方法
- 3.掌握 XSS 渗透的基本流程

2.1.2 背景知识

- 1.掌握 XSS 的基本原理
- 2.掌握 XSS 的分类和判断
- 3.掌握 XSS 的基本防御

2.1.3 实习时长

2 个学时

2.2 实训环境

kali linux 2024。

2.2.1 实训器材

kali linux 2024 虚拟机。

2.2.2 网络结构



kali linux 2024 虚拟机（）

本实验中，请实验老师根据学校组网与实验环境分配 IP。

2.3 实训内容

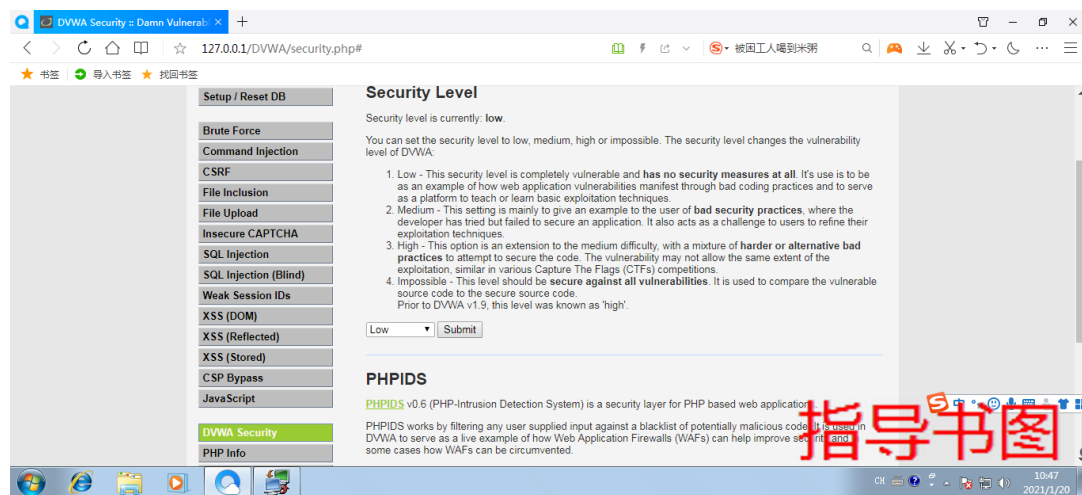
2.3.1 实训准备

打开 Apache2 和 mariadb 服务。

登录 DVWA，在浏览器输入 <http://127.0.0.1/DVWA/login.php>，转到 DVWA 的登录页面，使用用户名 admin，密码 password 进行登录。

2.3.2 反射型 XSS

1. DVWA 1.9 的代码分为四种安全级别：Low, Medium, High, Impossible。选择安全“Low”级别。



2. 反射型 XSS- Low

服务器端核心代码如下，可以看到，代码直接引用了 name 参数，并没有任何的过滤与检查，存在明显的 XSS 漏洞。

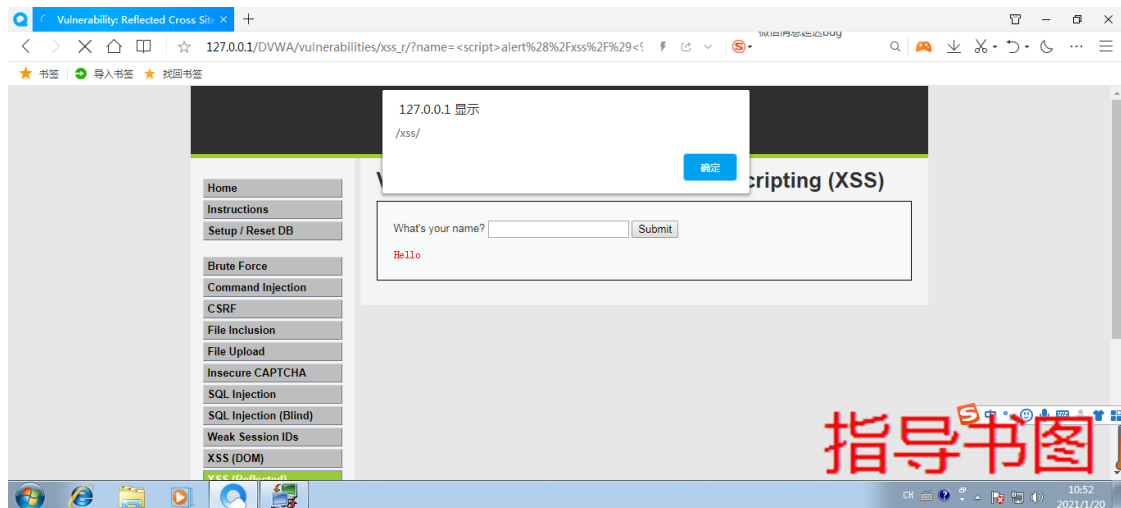
```
<?php
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {

    // Feedback for end user

    echo '<pre>Hello ' . $_GET[ 'name' ] . '</pre>';
}
?>
```

指导书图

输入<script>alert(/xss/)</script>，成功弹框：

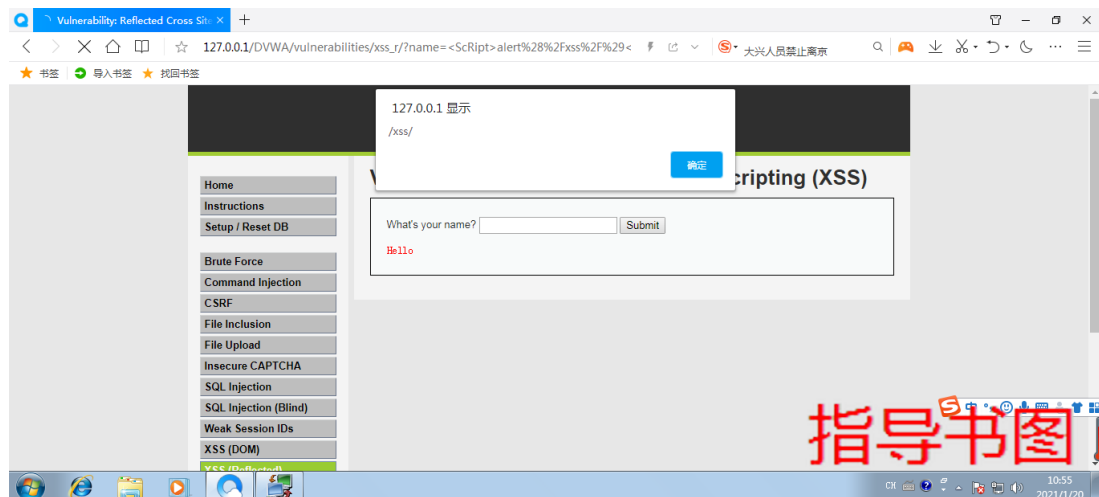


2. 反射型 XSS- Medium

服务器端核心代码如下，可以看到，这里对输入进行了过滤，基于黑名单的思想，使用 `str_replace` 函数将输入中的 `<script>` 删除。

```
<?php
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = str_replace( '<script>', '', $_GET[ 'name' ] );
    // Feedback for end user
    echo "<pre>Hello ${name}</pre>";
}
?>
```

输入 `<ScRipt>alert(/xss/)</script>`，大小写混淆绕过，成功弹框：



3. 反射型 XSS- High

服务器端核心代码如下，可以看到，High 级别的代码同样使用黑名单过滤输入，`preg_replace()` 函数用于正则表达式的搜索和替换，这使得双写绕过、大小写混淆绕过（正则表达式中 `i` 表示不区分大小写）不再有效。

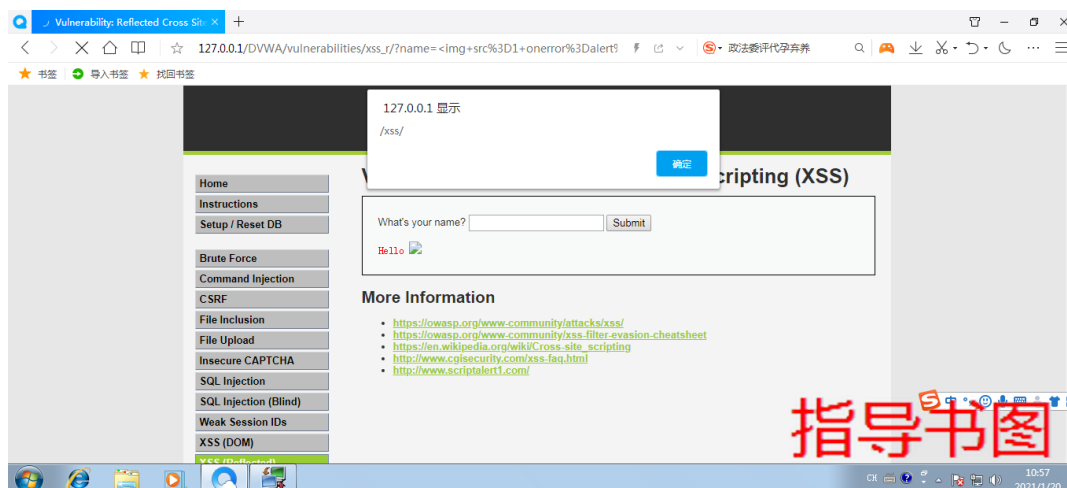
```

<?php
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = preg_replace( '/<(.*)(.*)c(.*)(.*)i(.*)(.*)p(.*)(.*)t/i', '', $_GET[ 'name' ] );
    // Feedback for end user
    echo "<pre>Hello ${name}</pre>";
}
?>

```

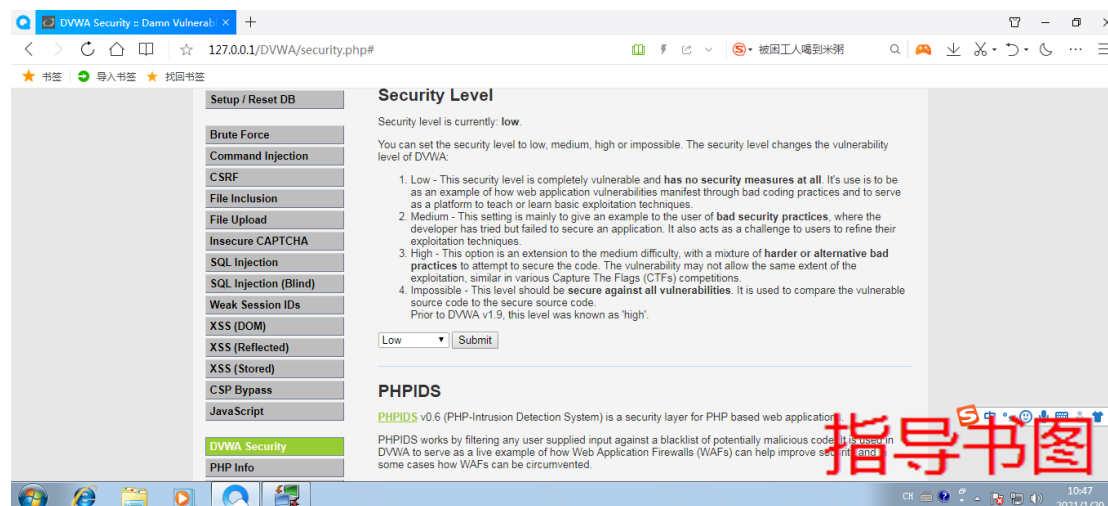
指导书图

虽然无法使用<script>标签注入 XSS 代码，但是可以通过 img、body 等标签的事件或者 iframe 等标签的 src 注入恶意的 js 代码。输入，成功弹框：



2.3.3 存储型 XSS

1. DVWA 1.9 的代码分为四种安全级别：Low, Medium, High, Impossible。选择“Low”安全级别。



2. 存储型 XSS- Low

服务器端核心代码如下，可以看到，对输入并没有做 XSS 方面的过滤与检查，且存储在数据库中，因此这里存在明显的存储型 XSS 漏洞。

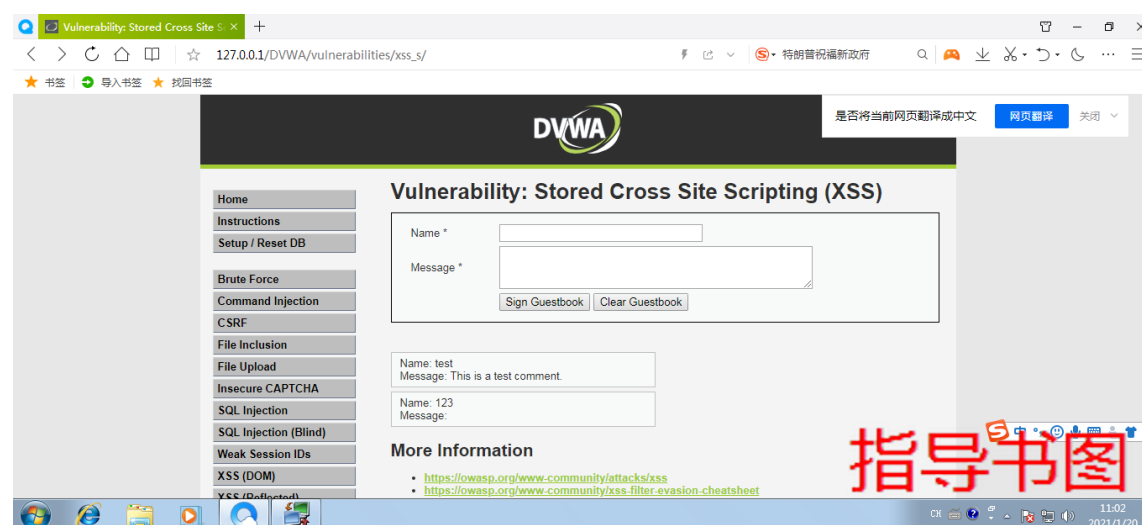
`trim(string,charlist)`: 函数移除字符串两侧的空白字符或其他预定义字符, 预定义字符包括、`\t`、`\n`、`\x0B`、`\r` 以及空格, 可选参数 `charlist` 支持添加额外需要删除的字符。

`mysql_real_escape_string(string,connection)`: 函数会对字符串中的特殊符号 (`\x00`, `\n`, `\r`, `\`, `'`, `"`, `\x1a`) 进行转义。

`stripslashes(string)`: 函数删除字符串中的反斜杠。

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name     = trim( $_POST[ 'txtName' ] );
    // Sanitize message input
    $message = stripslashes( $message );
    $message = mysql_real_escape_string( $message );
    // Sanitize name input
    $name = mysql_real_escape_string( $name );
    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' )";
    $result = mysql_query( $query ) or die( '<pre>' . mysql_error() );
    //mysql_close();
}
```

message 一栏输入`<script>alert(/xss/)</script>`, 成功弹框。



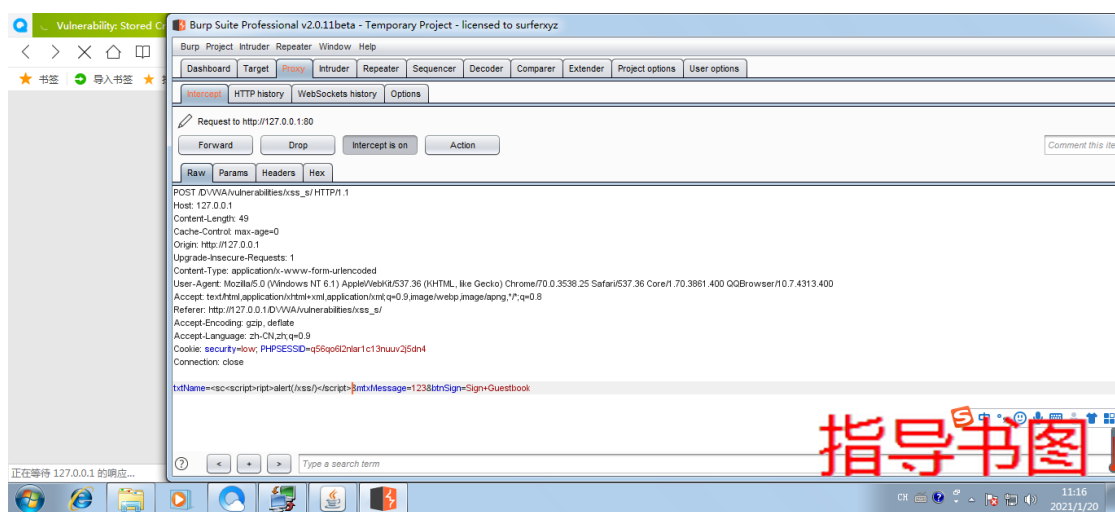
2. 存储 XSS- Medium

服务器端核心代码如下, 可以看到, 可以看到, 由于对 `message` 参数使用了 `htmlspecialchars` 函数进行编码, 因此无法再通过 `message` 参数注入 XSS 代码, 但是对于 `name` 参数, 只是简单过滤了 `<script>` 字符串, 仍然存在存储型的 XSS。

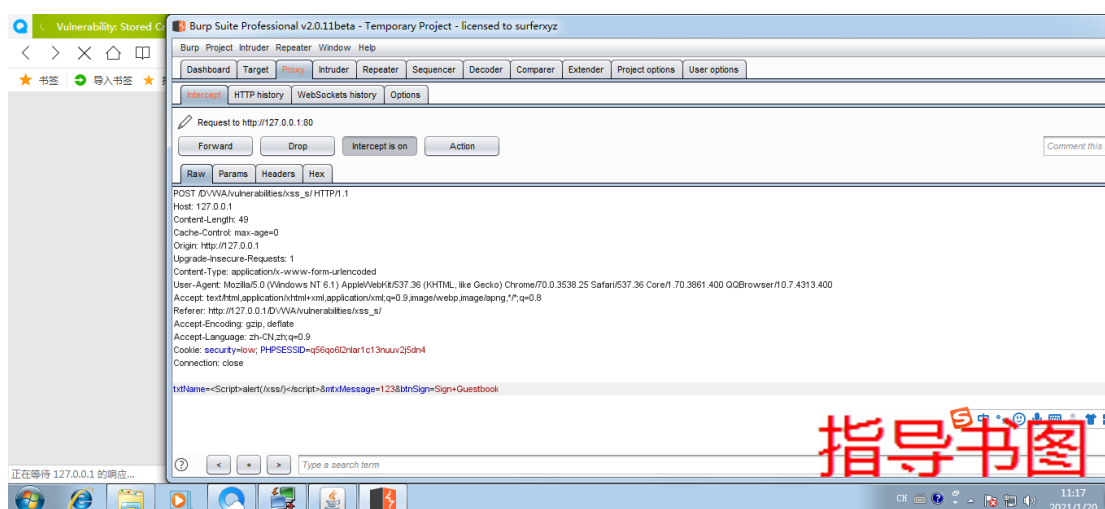
`strip_tags()` 函数剥去字符串中的 HTML、XML 以及 PHP 的标签, 但允许使用 `<b>` 标签。
`addslashes()` 函数返回在预定义字符 (单引号、双引号、反斜杠、NULL) 之前添加反斜杠的字符串。

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name     = trim( $_POST[ 'txtName' ] );
    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = mysql_real_escape_string( $message );
    $message = htmlspecialchars( $message );
    // Sanitize name input
    $name = str_replace( '<script>', '', $name );
    $name = mysql_real_escape_string( $name );
    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' )";
    $result = mysql_query( $query ) or die( '<pre>' . mysql_error() );
    //mysql_close();
}
?>
```

双写绕过：抓包改 name 参数为<sc<script>ript>alert(/xss/)</script>:



大小写混淆绕过：抓包改 name 参数为<Script>alert(/xss/)</script>:

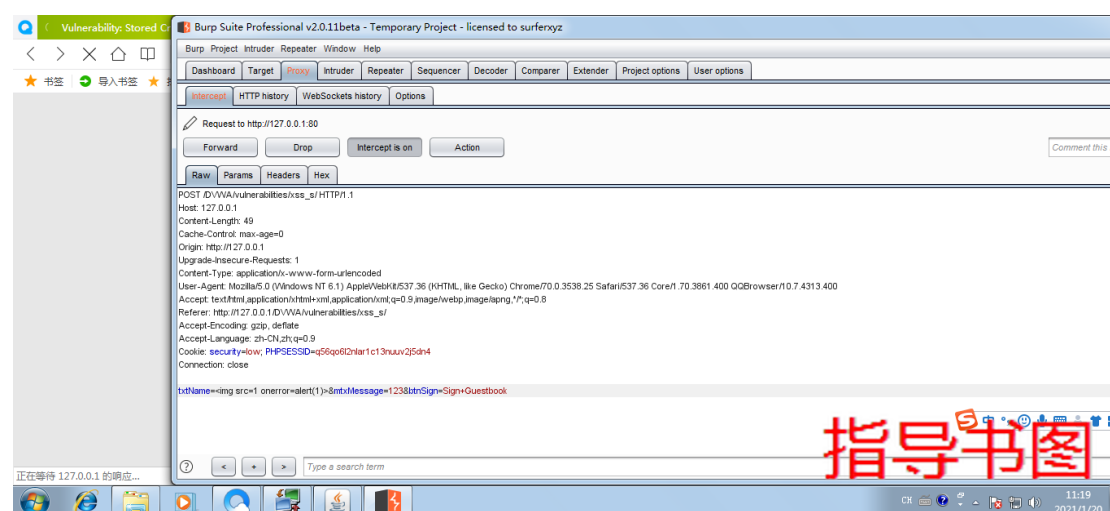


3. 存储型 XSS- High

服务器端核心代码如下，可以看到，可以看到，这里使用正则表达式过滤了<script>标签，但是却忽略了img、iframe 等其它危险的标签，因此 name 参数依旧存在存储型 XSS。

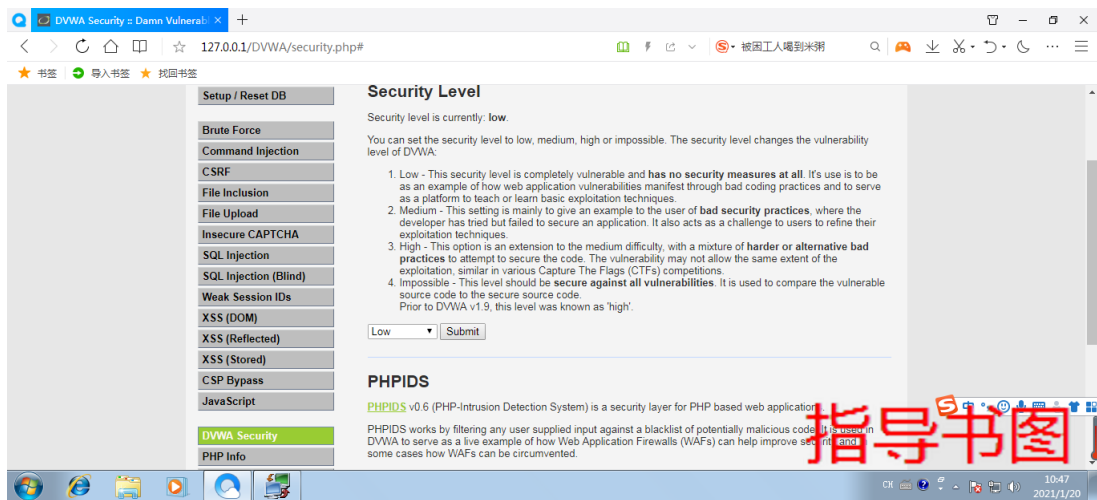
```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name     = trim( $_POST[ 'txtName' ] );
    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = mysql_real_escape_string( $message );
    $message = htmlspecialchars( $message );
    // Sanitize name input
    $name = preg_replace( '/<(.*?)s(.*?)c(.*?)r(.*?)i(.*?)p(.*?)t/i', '', $name );
    $name = mysql_real_escape_string( $name );
    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' )";
    $result = mysql_query( $query ) or die( '<pre>' . mysql_error() . '</pre>' );
    //mysql_close();
}
?>
```

抓包改 name 参数为:



2.3.4 DOM 型 XSS

1. DVWA 1.9 的代码分为四种安全级别: Low, Medium, High, Impossible。选择“Low”安全级别。

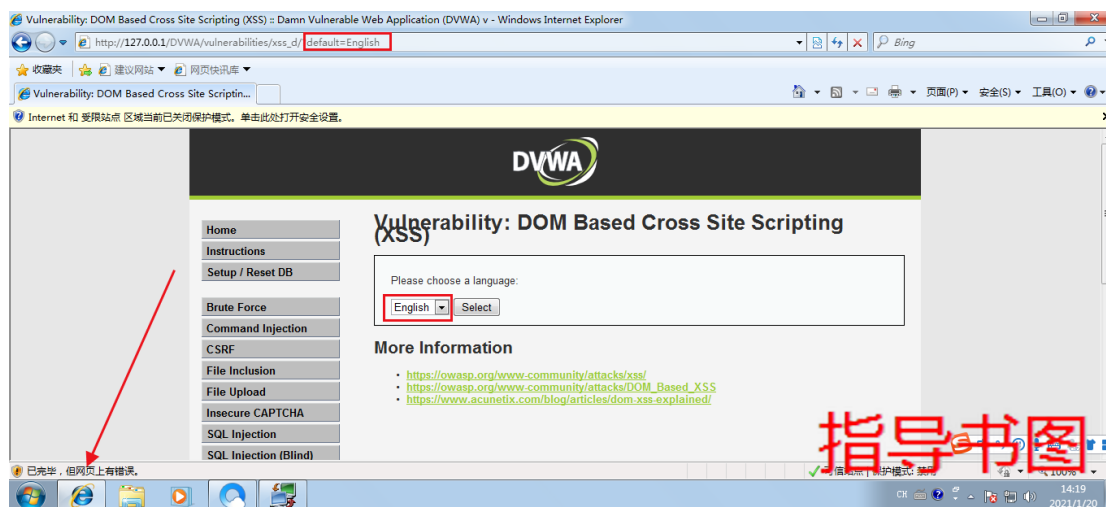


2. DOM 型 XSS- Low

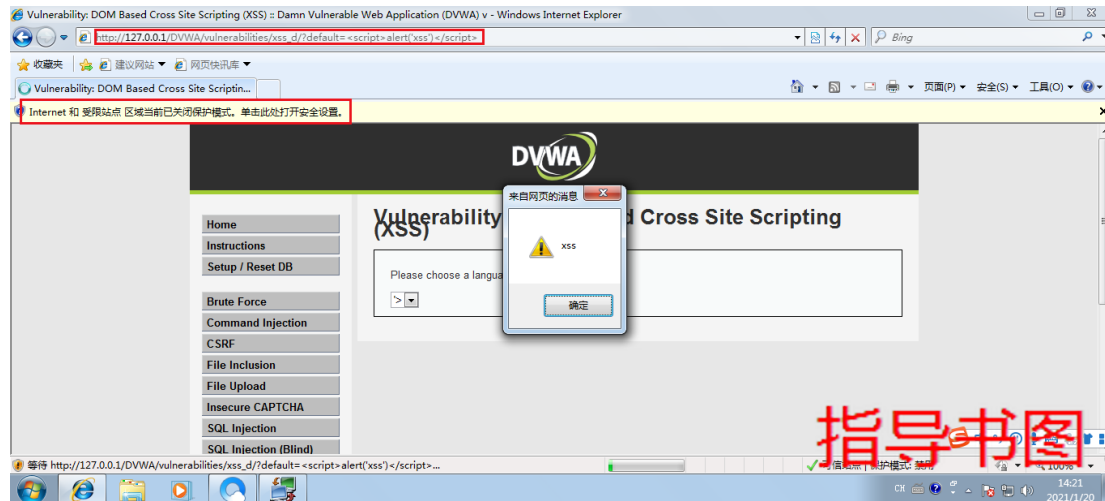
从源代码可以看出，这里 low 级别的代码没有任何的保护性措施！对 default 参数没有进行任何的过滤

```
<?php
# No protections, anything goes
?>
```

由于 QQ 浏览器做了防护，DOM 型 XSS 使用 IE 浏览器进行实验。



将 default 的参数 English 替换为<script>alert('xss')</script>，出现了弹窗。



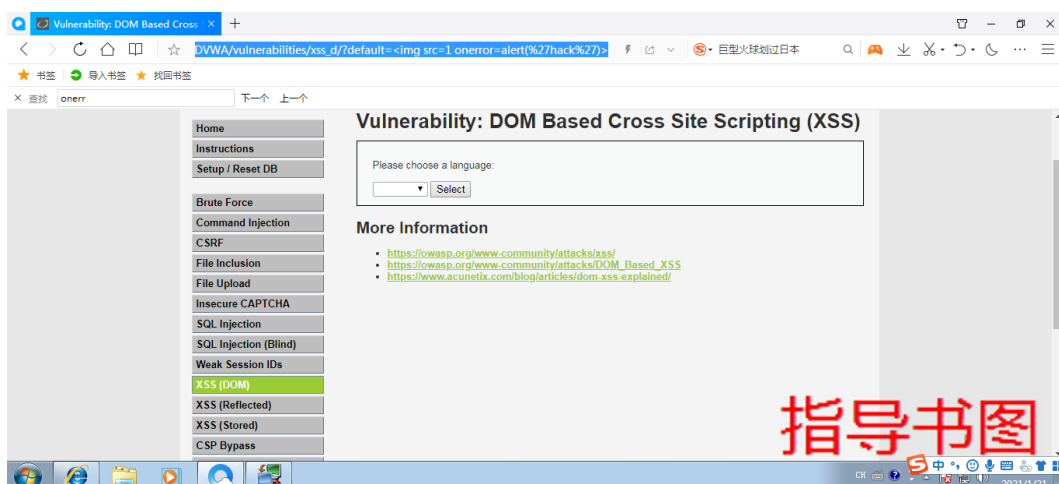
3. DOM 型 XSS- Medium

服务器端核心代码如下, 可以看到 medium 级别的代码先检查了 default 参数是否为空, 如果不为空则将 default 等于获取到的 default 值。这里还使用了 stripslashes 用于检测 default 值中是否有 <script>, 如果有的话, 则将 default=English。

```
<?php
// Is there any input?
if ( array_key_exists( "default", $_GET ) && !is_null ( $_GET[ 'default' ] ) ) {
    $default = $_GET['default'];

    # Do not allow script tags
    if (stripos ($default, "<script") != false) {
        header ("location: ?default=English");
        exit;
    }
}
?>
```

输入, 没有弹框:



查看源代码, 发现我们的语句被插入到了 value 值中, 但是并没有插入到 option 标签的值中, 所以 img 标签并没有发起任何作用。

```

▼<div class="body_padded">
  <h1>Vulnerability: DOM Based Cross Site Scripting (XSS)</h1>
  ▼<div class="vulnerable_code_area">
    <p>Please choose a language:</p>
    ▼<form name="XSS" method="GET">
      ▼<select name="default">
        ▶<script>...</script>
        <option value="%3Cimg%20src=1%20%CE%BFnerr%CE%BFr=alert(%27xss%27)%3E"></option>

```

指导书图

所以我们得先闭合前面的标签，我们构造语句闭合 option 标签：></option>，但是我们的语句并没有执行，于是我们查看源代码，发现我们的语句中只有 > 被插入到了 option 标签的值中，因为</option>闭合了 option 标签，所以 img 标签并没有插入。

```

▼<div class="body_padded">
  <h1>Vulnerability: DOM Based Cross Site Scripting (XSS)</h1>
  ▼<div class="vulnerable_code_area">
    <p>Please choose a language:</p>
    ▼<form name="XSS" method="GET">
      ▼<select name="default">
        ▶<script>...</script>
        <option value="%3E%3C/option%3E%3Cimg%20src=1%20%CE%BFnerr%CE%BFr=alert(%27xss%27)%3E"></option>

```

指导书图

继续构造语句去闭合 select 标签，</option></select>



指导书图

4. DOM 型 XSS- High

服务器端核心代码如下，可以看到白名单“只允许”传的“default 值”为 French、English、German、Spanish 其中一个。

```

<?php
// Is there any input?
if ( array_key_exists( "default", $_GET ) && !is_null ( $_GET[ 'default' ] ) ) {

    # White list the allowable languages
    switch ( $_GET[ 'default' ] ) {
        case "French":
        case "English":
        case "German":
        case "Spanish":
            # ok
            break;
        default:
            header ( "location: ?default=English" );
            exit;
    }
}

?>

```

指导书图

由于 form 表单提交的数据 想经过 JS 过滤 所以注释部分的 javascript 代码不会被传

到服务器端（也就符合了白名单的要求）。
`http://127.0.0.1/dvwa/vulnerabilities/xss_d/?default=English#<script>alert(/xss/)</script>`



2.4 总结与思考

2.4.1 实训总结

这部分内容是关于 XSS 攻击的操作过程，主要包括：

- 反射型 XSS 的利用方式
- 存储型 XSS 的利用方式
- DOM 型 XSS 的利用方式

2.4.2 思考题

问题 1：XSS 注入中主要有哪些限制？

问题 2：XSS 过滤只针对输入信息过滤有效吗？